

Topic 1: Variables and Data Types

Module: Python 101 • **Resource:** Teaching PDF

What you'll learn

This lesson introduces **variables** (names that refer to values) and the most common Python **data types**. You'll also see how Python represents data, when types matter, and how to convert between types safely.

Teaching-only resource (no exercises). Examples are for explanation and copy/paste reference.

Variables (names) vs values (objects)

In Python, variables don't "store" values directly. A variable is a **label** that refers to an object in memory.

```
x = 10
x = x + 5  # x now refers to a new int object (15)
```

Figure: Names point to objects (a simple mental model)

name: x	→	object: 10
reassign x	→	object: 15

Useful tools: **type(x)** tells you the type, and **id(x)** shows the identity (memory reference) for debugging identity questions.

Core data types (the everyday set)

Type	Example literal	What it's for	Common operations
int	42	Whole numbers	+, -, *, //, %, **
float	3.14	Decimal numbers	round(), math.*
bool	True / False	Logic / conditions	and, or, not
str	"hello"	Text	split(), replace(), f-strings
list	[1,2,3]	Ordered, mutable collection	append(), pop(), sort()
tuple	(1,2)	Ordered, immutable collection	packing/unpacking
dict	{"a": 1}	Key → value mapping	get(), keys(), values()



set	{1,2,3}	Unique items, fast membership	add(), union(), intersection()
-----	---------	-------------------------------	--------------------------------

Tip: Use **in** for membership checks (e.g., `"a" in my_dict` or `item in my_set`)—it reads clearly and is efficient.

Mutability (why it matters)

Some objects can be changed “in place” (**mutable**), and others cannot (**immutable**). This affects behavior when values are shared across variables.

Mutable (can change in place)	Immutable (cannot change in place)
list, dict, set	int, float, bool, str, tuple

```
a = [1, 2]
b = a
b.append(3)
# a is now [1, 2, 3] because a and b refer to the same list
```

If you need a copy: use **a.copy()** (shallow) or **copy.deepcopy(a)** (deep) depending on nested structures.

Type conversion (casting) & safe parsing

Python lets you convert types explicitly. The key is to convert at the boundary (inputs, files, APIs) and keep types consistent internally.

Goal	Example	Notes
String → int	<code>int("42")</code>	fails on "42.0"
String → float	<code>float("3.14")</code>	good for numeric input
Number → string	<code>str(123)</code>	useful for display/logging
Truthiness	<code>bool(x)</code>	empty/0 → False

```
raw = " 00123 "
value = int(raw.strip()) # 123
```

Figure: A safe input boundary pattern

Input (string)	→	Clean/validate	→	Convert	→	Use internally
----------------	---	----------------	---	---------	---	----------------

Quick reference: formatted text with f-strings

f-strings are the cleanest way to format values into strings:



```
name = "Ada"  
score = 93.5  
print(f"{name} scored {score:.1f}%")
```

Use format specifiers like **:.2f** (2 decimals), **,** (thousands separator), and alignment options for tables.